

Taller 3 – Documentación programación discreta

Jair Gómez Narváez, Aimer Esteban García Rojas

Programa de Ingeniería de Sistemas

Universidad Nacional de Colombia

Julio 2026

I. PROPÓSITO Y ALCANCE DEL DOCUMENTO

Este documento acompaña el repositorio de código correspondiente al Taller 3 de Matemáticas Discretas, para cada uno de los diez puntos propuestos en el enunciado se describe el problema que resuelve el programa correspondiente, la idea matemática en la que se apoya, la forma de ejecutarlo, las pruebas realizadas y las limitaciones identificadas.

El repositorio se organiza en cuatro carpetas dentro de src/cripto/ para los puntos 1 a 3, grafos/ para los puntos 4 a 6, boole/ para los puntos 7 a 9 y cuántica/ para el punto 10. Cada módulo puede ejecutarse de forma independiente y las pruebas automatizadas correspondientes se encuentran en la carpeta tests/.

II. BLOQUE A. CRIPTOGRAFÍA

II-A. Cifrado César: romper un mensaje antiguo

Problema. El programa cifra y descifra un texto desplazando cada letra una cantidad fija k de posiciones dentro del alfabeto y permite además recuperar un mensaje cuando no se conoce k , probando los 26 desplazamientos posibles.

Idea matemática. El cifrado se basa en aritmética modular sobre el alfabeto de 26 letras: representando cada letra por su posición $A=0, \dots, Z=25$, cifrar corresponde a la operación $(\text{posición} + k) \bmod 26$, y descifrar equivale a aplicar el desplazamiento inverso $-k$ sobre el mismo módulo. Al existir solo 26 claves posibles, el cifrado es trivialmente vulnerable a un ataque de fuerza bruta que las prueba todas.

Ejecución. El módulo src/cripto/cesar.py se ejecuta con python y despliega un menú de consola con tres opciones: cifrar, descifrar y fuerza bruta. Los caracteres que no son letras se conservan sin modificar y se respeta la diferencia entre mayúsculas y minúsculas.

Pruebas. Se verificó el caso del enunciado cifrar "HOLA UNAL" con $k = 3$ produce "KROD XQDO", la propiedad de que cifrar y luego descifrar con el mismo k reproduce el texto original y que el desplazamiento correcto aparece siempre entre las 26 opciones listadas por la fuerza bruta. Estas comprobaciones están automatizadas en tests/test_cesar.py.

Limitaciones. La implementación cubre únicamente el alfabeto A-Z, sin la letra Ñ y no ofrece ninguna seguridad real dado el reducido espacio de claves.

II-B. RSA de juguete: llaves, cifrado y descifrado

Problema.

Idea matemática.

Ejecución.

Pruebas.

Limitaciones.

II-C. MPC básico: calcular un promedio sin mostrar los datos

Problema. El programa simula un protocolo de computación multipartita segura con tres servidores, que permite calcular la suma y el promedio de una lista de notas sin que ningún servidor individual conozca los valores originales.

Idea matemática. Cada nota x se reparte en tres números aleatorios a, b, c tales que $(a + b + c) \bmod M = x$, con $M = 1000003$. Cada servidor recibe solo una de las tres partes de cada nota, de modo que ninguno puede reconstruir el valor original por separado; al sumar las tres partes parciales módulo M se recupera la suma total, de la que se obtiene el promedio.

Ejecución. El módulo src/cripto/mpc.py, ofrece la opción de revisar el ejemplo del enunciado o de ingresar una lista propia de notas enteras entre 0 y 50, mostrando la vista parcial que vería cada servidor y el resultado reconstruido.

Pruebas. El caso del enunciado, notas [40, 35, 50, 25] produce una suma de 150 y un promedio de 37.5, verificado con una aserción dentro del propio programa y en tests/test_mpc.py. También se probó el manejo de notas fuera de rango y de listas vacías, en ambos casos con un error controlado.

Limitaciones. El módulo M se fijó en 1000003, con listas más grandes o valores distintos habría que ajustarlo para

evitar colisiones. Además, el reparto aleatorio se simula dentro de un mismo proceso y no involucra una comunicación real entre servidores independientes.

III. BLOQUE B. GRAFOS

III-A. Ruta más corta: una ciudad como grafo

Problema. Encuentra la ruta de menor costo entre dos puntos de una red de transporte modelada como grafo ponderado, devolviendo tanto la distancia total como la secuencia de vértices que conforman la ruta.

Idea matemática. Se implementó el algoritmo de Dijkstra con una cola de prioridad, partiendo del origen con distancia 0, se expande en cada paso el vértice con menor distancia tentativa conocida. Como el grafo no tiene pesos negativos, una vez marcado un vértice como visitado su distancia ya es mínima, lo que garantiza un resultado acertado.

Ejecución. El módulo `src/grafos/dijkstra.py` carga un grafo de ejemplo de 8 vértices y 12 aristas desde un archivo CSV en `src/grafos/datos/` y permite ver el ejemplo predefinido, ingresar un origen y destino propios o abrir la interfaz gráfica `dijkstra_visual.py` para construir el grafo de forma interactiva.

Pruebas. Se verificó el caso de ejemplo Portal → Estadio, con distancia esperada 12 y ruta Portal→Museo→Calle26→Centro→Estadio, además de la consulta con vértices inexistentes, que produce un error controlado. Las pruebas están automatizadas en `tests/test_dijkstra.py`.

Limitaciones. El algoritmo asume pesos no negativos y no funcionaría correctamente si se introdujeran aristas de peso negativo. El grafo de prueba es pequeño y pensado para visualización, no se evaluó el rendimiento en redes de gran escala.

III-B. Cierre de una estación: medir el impacto en la red

Problema. Mide el impacto de cerrar un vértice o una arista de la red, por lo que recalcula las rutas más cortas entre varios pares origen destino antes y después del cierre y reporta cuáles aumentaron su distancia y cuáles quedaron desconectados.

Idea matemática. Se reutiliza el algoritmo de Dijkstra del punto anterior sobre dos versiones del grafo, la original y una copia a la que se elimina un vértice junto con todas sus aristas. Comparar las distancias par a par antes y después del cierre permite cuantificar el efecto estructural sobre la conexión de la red.

Ejecución. El módulo `src/grafos/cierre.py` expone las

funciones `eliminar_vertice()` y `eliminar_arista()`, que devuelve un grafo nuevo sin modificar el original, y una función `comparar_rutas()` que arma la tabla de resultados con origen, destino, distancia antes, distancia después, diferencia y estado.

Pruebas. Se probó el cierre de un vértice central, verificando que las rutas que lo atravesaban aumentaran su distancia o quedaran marcadas como desconectadas. Las pruebas están automatizadas en `tests/test_cierre.py`.

Limitaciones. El análisis se realiza par a par mediante ejecuciones independientes de Dijkstra, lo cual no escala bien si se quisiera evaluar el cierre de muchos vértices distintos sobre grafos de gran tamaño.

III-C. Coloreo de grafos: organizar exámenes sin choques

Problema. Asigna un color, representando una franja horaria a cada curso de modo que dos cursos con estudiantes en común, buscando que los vértices adyacentes nunca queden en la misma franja y reporta cuántos colores se usaron y qué vértices quedaron en cada uno.

Idea matemática. Se implementó un algoritmo voraz de coloreo, en la variante de Welsh Powell, se recorren los vértices y a cada uno se le asigna el primer color no usado por ninguno de sus vecinos ya coloreados. Esta estrategia siempre produce una asignación válida, aunque no garantiza el número mínimo de colores en el grafo.

Ejecución. El módulo `src/grafos/coloreo.py` carga un grafo de conflictos de al menos 10 vértices desde un archivo CSV, ejecuta el algoritmo voraz y verifica que ningún par de vértices adyacentes comparta color antes de mostrar el resultado agrupado por color.

Pruebas. Se verificó de forma automática que ningún par de vértices adyacentes recibiera el mismo color, sobre el grafo de prueba de mínimo 10 vértices interpretado como cursos con estudiantes cruzados. Las pruebas están en `tests/test_coloreo.py`.

Limitaciones. Al ser un algoritmo voraz, el número de colores usados depende del orden en que se procesan los vértices y no siempre corresponde al número mínimo teórico del grafo.

IV. BLOQUE C. ÁLGEBRA DE BOOLE, SHANNON Y COMPUTACIÓN CUÁNTICA

IV-A. Tablas de verdad y circuitos lógicos

Problema. Genera la tabla de verdad completa de una

expresión booleana con variables A, B y C, mostrando el resultado para cada una de las combinaciones posibles de entrada.

Idea matemática. Una expresión booleana de n variables define una función. Recorriendo todas las combinaciones posibles de entrada mediante producto cartesiano y evaluando la expresión en cada una, se obtiene la tabla completa, equivalente a la tabla de verdad de un circuito digital.

Ejecución. El módulo `src/boole/tablas.py` define como funciones de Python las expresiones exigidas en el enunciado, estas siendo $(A \wedge B) \vee (\neg C)$, $(A \oplus B) \wedge C$, $(A \vee B) \wedge (\neg A \vee C)$, se usa `itertools.product` para generar las 8 combinaciones de A, B y C e imprimir la tabla de cada una.

Pruebas. Se generaron las tablas completas de las tres expresiones obligatorias y se verificaron manualmente algunas filas contra la evaluación booleana esperada. Las pruebas automatizadas están en `tests/test_tablas.py`.

Limitaciones. La evaluación está implementada para expresiones fijas definidas como funciones de Python y no para un analizador de expresiones booleanas escritas como texto libre por el usuario.

IV-B. Simplificación booleana: hacer un circuito más barato

Problema.

Idea matemática.

Ejecución.

Pruebas.

Limitaciones.

IV-C. Shannon: medir información en un mensaje

Problema.

Idea matemática.

Ejecución.

Pruebas.

Limitaciones.

IV-D. Primer simulador cuántico: bits, qubits y mediciones

Problema.

Idea matemática.

Ejecución.

Pruebas.

Limitaciones.
